Listen

Week 13 Lab: Data Analysis for Security

What Are We Building?

Security teams don't just find vulnerabilities — they need to **analyze** patterns. Which subdomains have the most issues? What severity level dominates? Today you'll use Python's built-in `csv` module to import scan data and generate useful reports and visualizations.

Open the starter files and look for the **`# TODO`** markers.

Q1: Import and Analyze Scan Results

File: [lab13_q1_starter.py]

Your team has been scanning `0x10.cloud` and has a CSV file of all findings. You'll write functions to load, filter, count, and rank the data.

The starter creates a sample CSV and has the main section. **Your work is four functions.**

`load_findings(filename)` — read the CSV using `csv.DictReader`, return a list of dictionaries

`count_by_field(findings, field)` — count occurrences of each unique value in a field

`filter_findings(findings, field, value)` — list comprehension to filter by field value

`top_subdomains(findings, n)` — find the n most targeted subdomains

Reference

Python

```python
import csv

# Read CSV as list of dictionaries
with open("data.csv", "r") as f:
    reader = csv.DictReader(f)
    rows = list(reader)
# rows[0] → {"subdomain": "ssh.0x10.cloud", "type": "default_creds", ...}

# Count occurrences
counts = {}

```

```python
    for row in rows:
        val = row["severity"]
        counts[val] = counts.get(val, 0) + 1

    # Sort dictionary items by value
    sorted(counts.items(), key=lambda x: x[1], reverse=True)
```

Test It

Run `python lab13_q1_starter.py`. You should see severity counts, type counts, HIGH findings listed, and top 3 subdomains.

Q2: ASCII Dashboard

File: [`lab13_q2_starter.py`]

Instead of just numbers, you'll create visual bar charts using ASCII characters — how many command-line security tools present data.

The starter has sample data and main section. **Your work is three functions.**

`bar_chart(data, title, max_width=30)` — draw horizontal bars using `█` characters, scaled to max value

`severity_summary(findings)` — count per severity, return as [(severity, count)] ordered HIGH first

`timeline(findings)` — count per date, return as [(date, count)] sorted by date

Reference

Python

```python
# Repeating a character
"█" * 15    # "└──────────────┘"

# Scaling a value to a max width
bar_length = int((count / max_value) * max_width)

# Sorting a dictionary by value
sorted(counts.items(), key=lambda x: x[1], reverse=True)
```

Test It

Run `python lab13_q2_starter.py`. You should see three bar charts: severity breakdown, findings by date, and vulnerability types.

Submission

- Only submit your GitHub (No Screenshot, no ZIP or Python file, just the Link) repository link, for example: https://github.com/sojoudian/COMP2152_Labs
- Create a branch named `lab_week13`, complete your lab work in that branch, and once finished, merge it into your `master` branch.